Midterm Progress Report (Track D, implementation-based): Do Generative Level Models Copy Their Training Data?

Sujal Shrestha SUJALSHRESTHA@OU.EDU
Jackson McKay JACKSON.J.MCKAY-1@OU.EDU
CS 4713, Fall 2026
Github: https://github.com/sujalstha/MarioGPT

Abstract

A playable generated Mario level may resemble a training level. We ask how often small Transformer and diffusion generators produce near-copies, and whether copying decreases as the training set grows. We trained 50 models on matched subsets of 3, 6, 9, 12, or 15 levels, with five seeds per setting, and generated 1,000 slices per model. At three levels, mean near-copy rates were 25.5% for the Transformer and 23.9% for diffusion. These are measurements from short, fixed training budgets rather than converged reproductions. Low copying alone cannot establish generalization: our held-out baseline and local reachability check help distinguish novelty from poor generation.

Keywords: Track D, theory of modern AI; implementation-based study; memorization, level generation

1. Introduction

Zhong (2025) identifies gaps in evaluating generated game content (Section 4.3). Our copy-rate study addresses one part of that broader evaluation problem.

Our proposal compares MarioGPT (Sudhakaran et al., 2023), a DistilGPT2-based tile generator, with the attention-equipped U-Net diffusion generator of Lee and Simo-Serra (2023). Both evaluate playability with an A* agent. MarioGPT already examines memorization in Section 4.3 through nearest-training examples and sampling temperature. We extend that question with a quantitative copy threshold and a matched training-set-size sweep.

Our hypothesis is that more than half the outputs are near-copies at three training levels, and that copying falls as data increases. This tests overfitting behavior rather than proving a generalization bound.

2. Implementation and Results

Setup. We use 37 VGLC maps from Super Mario Bros. and The Lost Levels. Five whole levels are held out before extracting 14×16 slices (stride 1); maps are ground-aligned by cropping or padding above. Five seeds give nested subsets $k \in \{3, 6, 9, 12, 15\}$ shared by both architectures. Each of the $2 \times 5 \times 5 = 50$ models generates 1,000 slices. The private repository records data IDs, settings, outputs, and losses; `mlciv` has been invited.

The Transformer retains pretrained DistilGPT2 blocks with a tile vocabulary, omitting prompts and novelty search. It trains 300 updates with AdamW (5×10^{-5}, zero weight decay), sampling at temperature 1 without top-k truncation. Our 32/64/32-channel U-Net omits attention, training 1,500 updates with Adam (10^{-3}) on noise-prediction MSE. It uses 100 linear-beta DDPM steps (10^{-4} to

© 2027 S. Shrestha & J. McKay.

Table 1: Near-copy rate (%): mean ± sample standard deviation over five seeds. The held-out baseline is shared by both architectures.

Training levels	Transformer	Diffusion	Held-out real slices
3	25.5 ± 12.8	23.9 ± 13.8	27.9 ± 3.1
6	32.4 ± 19.4	23.3 ± 10.3	35.7 ± 1.8
9	29.4 ± 11.0	38.9 ± 26.4	37.7 ± 2.5
12	40.0 ± 14.9	37.8 ± 29.5	39.2 ± 3.0
15	31.1 ± 6.7	29.8 ± 22.0	40.7 ± 2.5

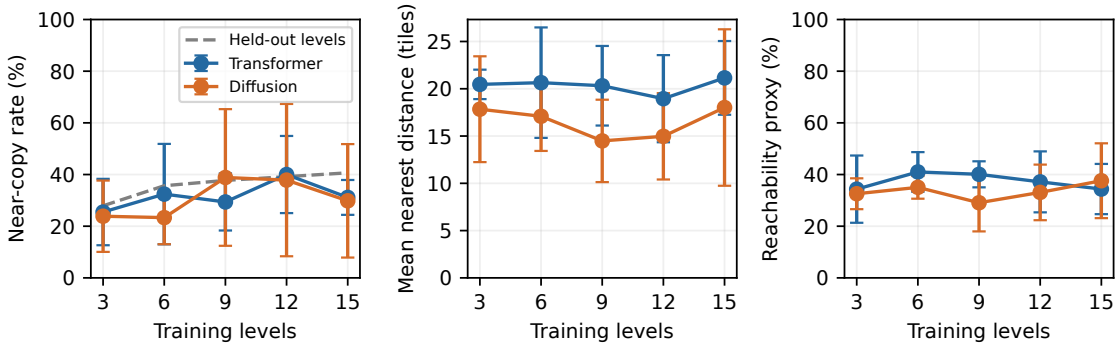


Figure 1: Copy rate, nearest-training distance, and local reachability versus training-set size. Error bars show sample standard deviation over five seeds. The dashed curve is the held-out real-level baseline.

0.1) and argmax decoding. Both use batch 16. These are small adaptations; the diffusion paper uses 14×14 slices. Budgets are fixed across dataset sizes within each architecture.

Experiment Set 1: copying. We compute exact nearest-training Hamming distance, $d_{\min}(x) = \min_{s \in S} \sum_{i=1}^{224} \mathbf{1}[x_i \neq s_i]$. Near-copies have $d_{\min} \leq 11$ (at most 5% of tiles); exact copies have distance zero. Table 1 and Figure 1 show results across five training runs per setting. Held-out real slices provide a baseline because larger training corpora offer more possible matches. Generated outputs are not independent training repeats.

Experiment Set 2: filtering. A point-agent search tests left-to-right reachability using supported landing positions and approximate jump arcs (horizontal range and height up to four tiles). Enemies block traversal. We repeat the check on non-copied outputs. This is a local proxy, not Mario physics or full-level A* playability. Figure 2 shows minimum-distance outputs at $k = 3$, seed 0.

Transformer: mean reachability 37.4% before filtering and 35.4% after; exact-copy rate 1.0%. Diffusion: mean reachability 33.5% before filtering and 37.0% after; exact-copy rate 0.0%. Means weight the 25 runs equally. Both three-level mean copy rates are below our hypothesized 50%, and neither curve decreases monotonically. Short training and unequal capacities limit interpretation: low copying does not establish useful generalization.

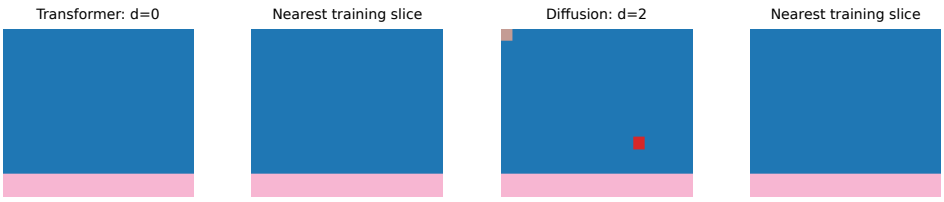


Figure 2: Closest generated/training pairs ($k = 3$, seed 0). Blue is empty space; pink is solid X blocks. Common ground-and-sky patterns can match exactly.

3. Difficulties So Far

- **Data:** Lost Levels supplements the 15-map base corpus; mixed heights require normalization.
- **Compute:** The workflow used 17:12 GPU-minutes (cap: 45). These short runs do not establish convergence.
- **Evaluation:** Common patterns inflate matches. Hamming boundary and simple reachability tests passed; all saved distances were recomputed.

4. Plan for the Rest of the Semester

- **Nov. 10–25:** Check convergence with longer training, vary sampling temperature, and improve playability evaluation on the same splits.
- **Nov. 26–Dec. 7:** Relate copying to training exposure, capacity, and overfitting.
- **Dec. 8/10; Dec. 14:** Present, then submit the final report and code. The required 15-minute meeting with Dr. Cao remains to be scheduled and completed.

5. What Each of Us Has Done

Sujal (Transformer half)

- Read MarioGPT and summarize its model, data and evaluation.
- Build the DistilGPT2 tile generator and run its 25 models (5 dataset sizes $\times$ 5 seeds).
- Write the copy-rate metric (nearest-neighbor Hamming distance) and compute the held-out baseline.
- Make Table 1 and Figure 2 (the closest generated/training pairs).
- Write the Introduction, the Transformer part of Setup, and Experiment Set 1.

Jackson (diffusion half)

- Read the diffusion paper and summarize it.
- Build the U-Net DDPM and run its 25 models.
- Prepare the VGLC data: download, hold out 5 levels, cut slices, make the nested training subsets.

- Write the reachability check and the filtering experiment.
- Make Figure 1 (copy rate, distance and reachability plots).
- Write the diffusion part of Setup, Experiment Set 2, and Difficulties.

AI usage. Used Claude Opus 5.5

Sujal used AI to understand how MarioGPT learns by predicting the next tile in a sequence, and why that training goal tends to reward copying. He also used it to learn how the temperature setting changes how random or copy-like the output is, and how to compute the copy metric efficiently (counting mismatched tiles against every training slice, with 11 of 224 as the near-copy cutoff). Finally, he used it to check that the statistics are right: averaging over 5 training runs, and why the 1,000 slices from one model aren't independent samples.

Jackson used AI to understand how a diffusion model adds noise to levels step by step and learns to predict and remove it, including the math of the noise schedule and the training loss. He also used it to learn how generation runs that process in reverse over 100 steps, why turning tiles into one-hot vectors and rounding back is an approximation, and how to frame the playability check as a path search problem.

References

S. Sudhakaran, M. González-Duque, M. Freiberger, C. Glanois, E. Najarro, and S. Risi. MarioGPT: Open-Ended Text2Level Generation through Large Language Models. *NeurIPS*, 2023.

H. J. Lee and E. Simo-Serra. Using Unconditional Diffusion Models in Level Generation for Super Mario Bros. *MVA*, 2023.

Chenhui Zhong. A Comprehensive Review of Transformer and Diffusion Models in Game Design: Applications, Challenges, and Future Directions. In *Proceedings of the 5th International Conference on Signal Processing and Machine Learning*, pages 135–141, 2025. DOI: https://doi.org/10.54254/2755-2721/133/2025.20611.